

ISEN/DAEN 427 Lecture 6

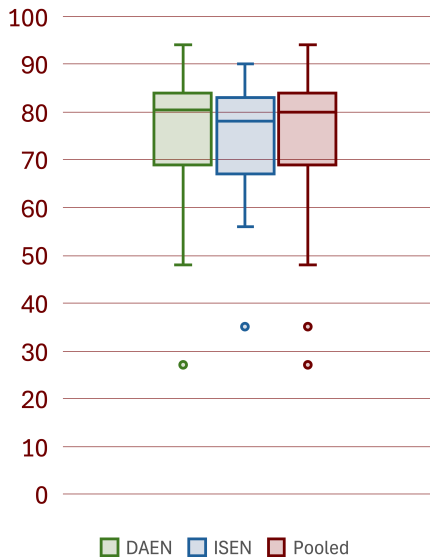
Alexander Abuabara

Computational Aspects of Bayesian Inference

BCP Ch. 1



Midterm 1



Computational Challenge

- Recall the general form of Bayes rule: the posterior distribution is proportional to the product of the *likelihood* and a *priori* distributions:

$$\pi(s) := \mathbb{P}(s|x) = \kappa \times \mathbb{P}(x|s)\mathbb{P}(s)$$

- Since $\int \pi(s)ds = 1$, the normalizing constant is:

$$\frac{1}{\kappa} = \int \mathbb{P}(x|s)\mathbb{P}(s)ds = \mathbb{P}(x)$$

but computing this integral is exceedingly costly.

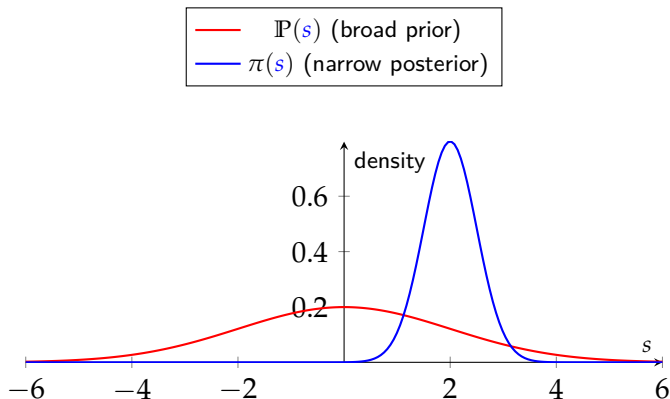
Monte Carlo Estimation

- One approach is to sample from $\mathbb{P}(s)$ say $s_1, s_2, \dots, s_N$ and estimate the normalizing constant by $\hat{\kappa} := \frac{1}{\hat{\gamma}}$ where:

$$\hat{\gamma} = \frac{1}{N} \sum_{n=1}^N \mathbb{P}(x|s_n) \mathbb{P}(s_n)$$

- If the posterior distribution $\pi(s)$ is much more concentrated than $\mathbb{P}(s)$, then many samples s_n will have *low* likelihood values $\mathbb{P}(x|s_n)$. Thus, the standard error of $\hat{\gamma}$ can be large.

Illustration



Posterior $\pi(s)$ (blue) is highly concentrated near $s = 2$.
Most samples s_n fall there, where the prior $\mathbb{P}(s)$ (red) is small.

- Posterior $\pi(s)$ concentrated $\Rightarrow$ samples from a small region.
- Prior $\mathbb{P}(s)$ broad $\Rightarrow$ that region has low prior density.
- Many samples have low likelihood $\mathbb{P}(x|s_n)$.
 $\Rightarrow$ large standard error

If posterior is too narrow compared to prior,
importance sampling becomes inefficient.

Problem:

To obtain samples from $\pi(s), s \in S$ but we only know how to compute $\frac{\pi(s_i)}{\pi(s_j)}$ for $s_i, s_j \in S$.

Problem:

To obtain samples from $\pi(s), s \in S$ but we only know how to compute $\frac{\pi(s_i)}{\pi(s_j)}$ for $s_i, s_j \in S$.

Solution: Construct a Markov chain $\{X_n : n \geq 0\}$ with state space S and limit distribution $\pi(s), s \in S$.

Simulate Markov-chain so that in the long-run, samples $\{X_n : n \geq 0\}$ are *approximately* distributed according to $\pi(s)$.

Metropolis-Hastings Algorithm

- Let $\tilde{\mathbf{P}}$ be *any* **symmetric**, **ergodic** one-step transition probability matrix on the state space S .
 - Ergodicity: the Markov chain must be able to reach any state from any other state in a finite number of steps.
 - Balanced: the prob. of being in a state x and moving to x' must be the same as being in x and moving to x .
- Given the current state $X_n = s_i$:
 - Draw a sample $s_j \sim \tilde{\mathbf{P}}(s_i, \cdot)$
 - If $s_j \neq s_i$, accept sample with probability

$$\alpha(s_i, s_j) = \begin{cases} 1 & \text{if } \frac{\pi(s_j)}{\pi(s_i)} \geq 1 \\ \frac{\pi(s_j)}{\pi(s_i)} & \text{if } \frac{\pi(s_j)}{\pi(s_i)} < 1 \end{cases} = \min\left\{1, \frac{\pi(s_j)}{\pi(s_i)}\right\}$$

that is:

$$X_{n+1} = \begin{cases} s_j & \text{with probability } \alpha(s_i, s_j) \\ s_i & \text{with probability } 1 - \alpha(s_i, s_j) \end{cases}$$

Metropolis-Hastings Algorithm

The one-step probability distribution of the *induced* Markov chain is:

$$\mathbf{P}(s_i, s_j) = \begin{cases} \tilde{\mathbf{P}}(s_i, s_j)\alpha(s_i, s_j) & s_j \neq s_i \\ 1 - \sum_{s_j \neq s_i} \tilde{\mathbf{P}}(s_i, s_j)\alpha(s_i, s_j) & s_j = s_i \end{cases}$$

Metropolis-Hastings Algorithm

The resulting Markov chain is **reversible**, i.e. for $s_j \neq s_i$

$$\begin{aligned}\pi(s_i)\mathbf{P}(s_i, s_j) &= \pi(s_i)\tilde{\mathbf{P}}(s_i, s_j)\alpha(s_i, s_j) \\ &= \tilde{\mathbf{P}}(s_i, s_j)\pi(s_i) \min\left\{1, \frac{\pi(s_j)}{\pi(s_i)}\right\} \\ &= \tilde{\mathbf{P}}(s_i, s_j) \min\{\pi(s_i), \pi(s_j)\} \\ &= \tilde{\mathbf{P}}(s_j, s_i)\pi(s_j) \min\left\{1, \frac{\pi(s_i)}{\pi(s_j)}\right\} \\ &= \pi(s_j)\mathbf{P}(s_j, s_i)\end{aligned}$$

Metropolis-Hastings Algorithm

- Recall that the long-run distribution μ of an ergodic Markov chain with transition probabilities $\mathbb{P}(s_i|s_j)$ satisfies:

$$\sum_{s_j \in S} \mu(s_j) \mathbb{P}(s_i|s_j) = \mu(s_i) \quad \forall i \in S$$

- For the induced Markov chain

$$\begin{aligned} \sum_{s_j \in S} \pi(s_j) \mathbf{P}(s_j, s_i) &= \sum_{s_j \in S} \pi(s_i) \mathbf{P}(s_j, s_i) \\ &= \pi(s_i) \sum_{s_j \in S} \mathbf{P}(s_i, s_j) \\ &= \pi(s_i) \end{aligned}$$

Metropolis-Hastings Algorithm

- In conclusion, we are able to obtain samples $\{X_n : n > 0\}$ with limiting (long-run) distribution equal to $\pi(\cdot)$, i.e.

$$\lim_{n \rightarrow \infty} \mathbb{P}(X_n = s) = \pi(s)$$

for all $s \in S$.

Metropolis-Hastings Algorithm

- In conclusion, we are able to obtain samples $\{X_n : n > 0\}$ with limiting (long-run) distribution equal to $\pi(\cdot)$, i.e.

$$\lim_{n \rightarrow \infty} \mathbb{P}(X_n = s) = \pi(s)$$

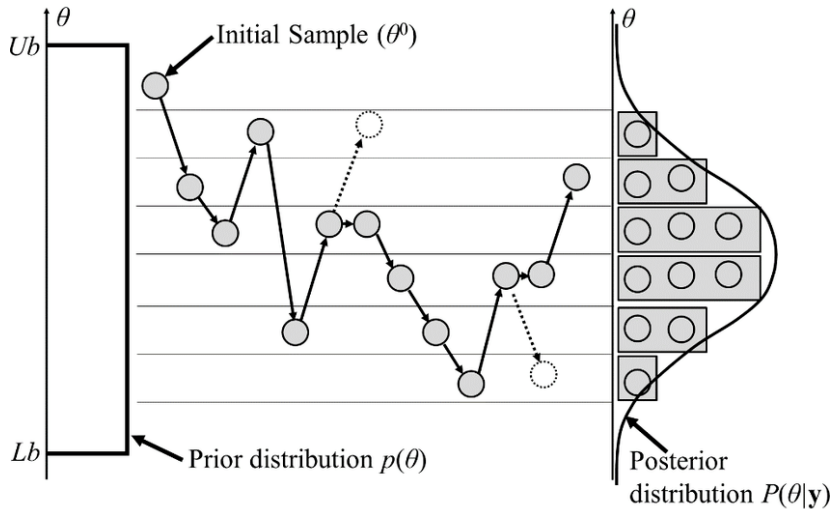
for all $s \in S$.

- This is done by *only* relying on the knowledge of **posterior** likelihood ratio: $\frac{\pi(s_i)}{\pi(s_j)}$.

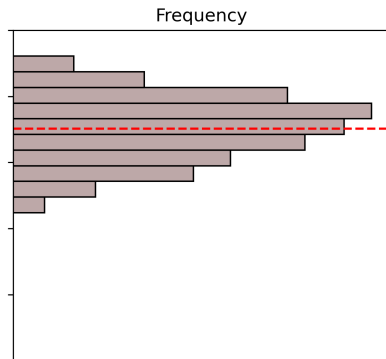
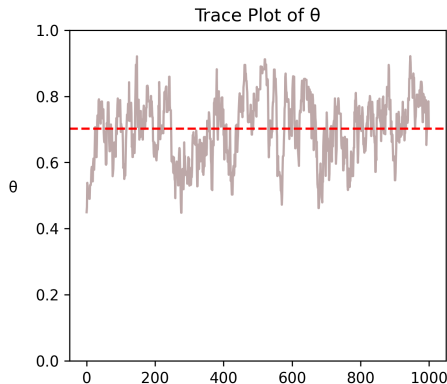
Metropolis-Hasting Algorithm

- ① Initialization: Choose an arbitrary point x_t and propose a function $g(x | y)$ that satisfies $g(x | y) = g(y | x)$
- ② For each iteration t :
 - Propose a candidate x' for the next sample by picking from the distribution $g(x' | x_t)$
 - Calculate the acceptance ratio $\alpha = f(x')/f(x_t) = P(x')/P(x_t)$ (f is proportional to the density of P)
 - Accept or reject:
 - Generate a uniform random number $u \in [0, 1]$
 - If $u \leq \alpha$ then accept the candidate by setting $x_{t+1} = x'$
 - If $u > \alpha$ then reject the candidate and set $x_{t+1} = x_t$ instead

Metropolis-Hastings Algorithm

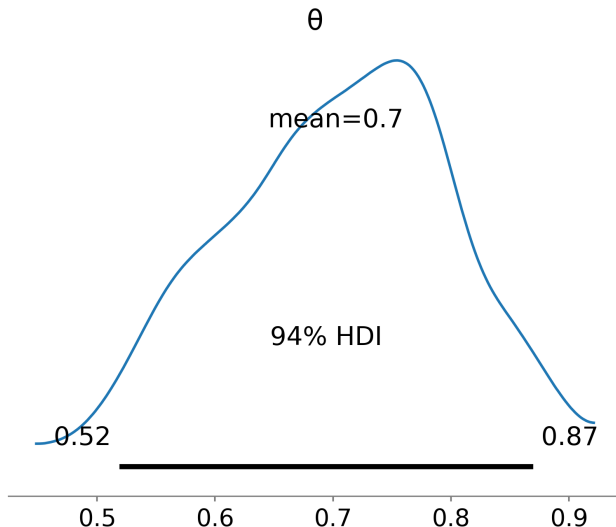


Metropolis-Hasting Algorithm DIY

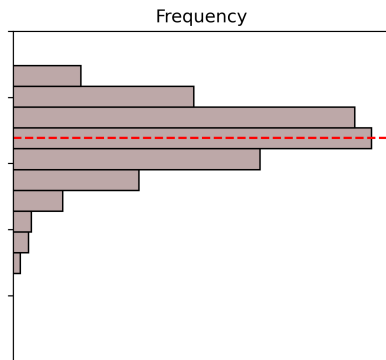
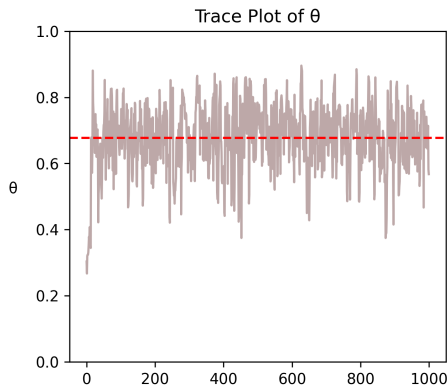


Trace of MCMC Simulation

Metropolis-Hasting Algorithm DIY

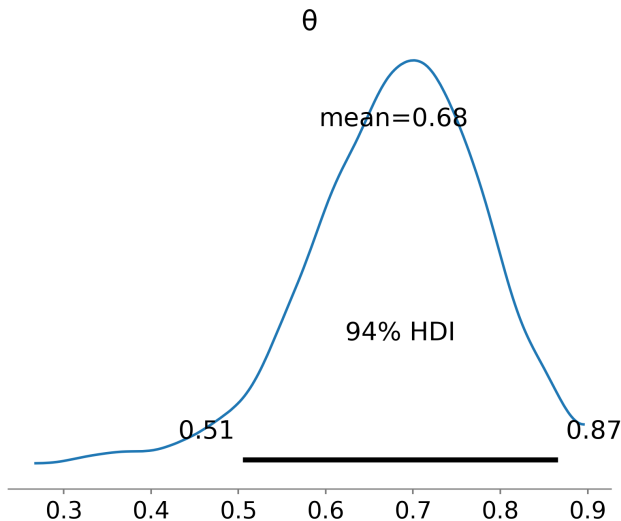


Metropolis-Hasting Algorithm PyMC



Trace of PyMC Simulation

Metropolis-Hasting Algorithm PyMC



Python Implementation

```
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

def posterior(theta, Y, alpha, beta):
    if 0 <= theta <= 1:
        prior = stats.beta(alpha, beta).pdf( )
        like  = stats.bernoulli(theta).pmf(Y).prod()
        prob = like * prior
    else:
        prob = -np.inf
    return prob
```

Python Implementation

```
Y = stats.bernoulli(0.7).rvs(20)

n_iters = 5000
candidate_sd = 0.05
alpha = beta = 1
theta = 0.5
trace = {"theta": np.zeros(n_iters)}
p_i = posterior(theta, Y, alpha, beta)

for iter in range(n_iters):
    theta_candidate = stats.norm(theta, candidate_sd).rvs(1)
    p_j = post(theta_candidate, Y, alpha, beta)
    pa = p_j / p_i
    if pa > stats.uniform(0, 1).rvs(1):
        theta = theta_candidate
        p_i = p_j
    trace["theta"][iter] = theta
```

Using PyMC

```
import pymc as pm
import arviz as az

from pymc import Beta, Model, Binomial, sample

# Declare a model in PyMC
with Model() as model:
    # Specify the prior distribution of unknown parameter
    theta = Beta("theta", alpha=1, beta=1)

    # Specify the likelihood distribution and condition on the observed data
    y_obs = Binomial("y_obs", n=1, p=theta, observed=Y)

    # Sample from the posterior distribution
    idata = sample(1000, return_inferencedata=True)
```
